JOHNS HOPKINS
WHITING SCHOOL of ENGINEERING

Introduction to Generative AI

Mathematical Model of a Transformer

Outline

- Module Learning Objectives
- Sequence-to-Sequence Models
- Introduction to Transformer NN Models
- Attention Mechanism

Module Learning Objectives

- Review of the neural network architecture and mathematical model of an attention mechanism
- Explain the significance of transformers in modeling sequential data and their advantages over RNNs
- Compare various attention models, including Bahdanau, Luong, and Multi-Head Attention
- Analyze the mathematical models underlying attention mechanisms and positional encoding
- Demonstrate the use of transformers in sequence-to-sequence tasks such as language translation

Sequence-to-Sequence Models

- A set of N many data points $x_i \in \mathbb{R}^{\mathrm{M}}$ in M dimensions are assumed to have independent features with no dependence on their order (i.e. a shuffle is OK)
- When the knowledge is carried by a sequence of data points $\langle x_t \rangle$, such as speech, video, text, events on a network, etc., then we deal with a timeseries or sequence
- Sequences can be represented naturally on the directed graphs of RNNs
- **Encoder-Decoder Framework** The input $\mathbf{X}$ is used to generate the hidden states

  $$h_t = f_{enc}(x_t, h_{t-1}; \theta_{enc})$$

  by the encoder E can be used by the decoder D output states and context vector

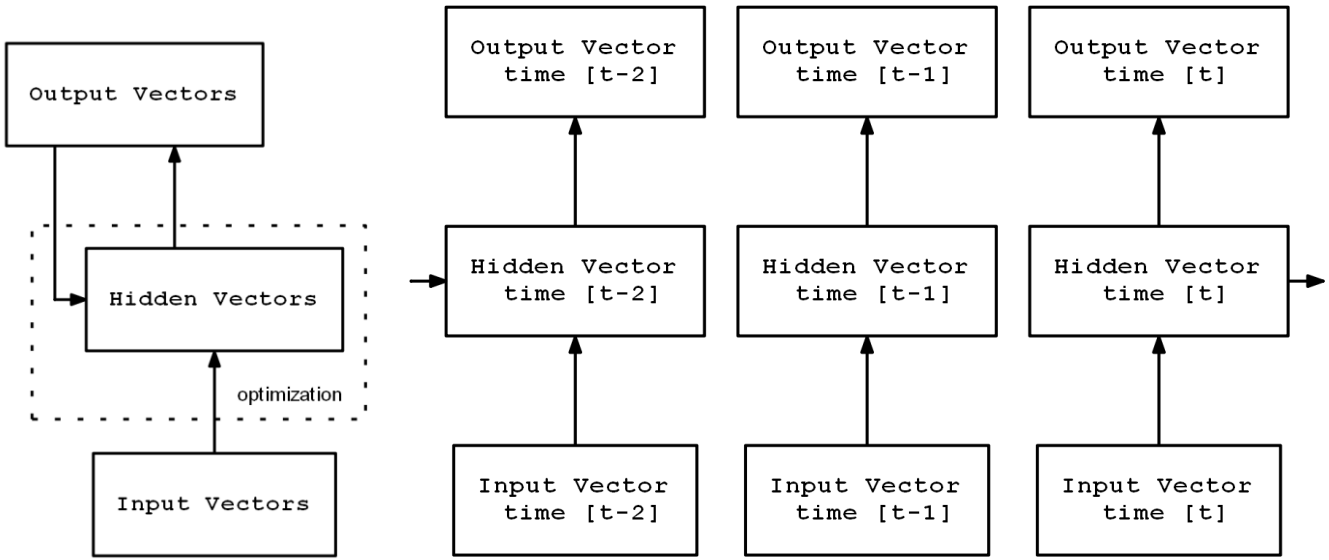
  $$s_t = f_{dec}(y_{t-1}, s_{t-1}, c_t; \theta_{dec})$$

  to generate an output sequence $\mathbf{Y}$, where the context vector is given by

  $$c_t = \sum_{i=1}^{T} \alpha_{t,i} h_i$$

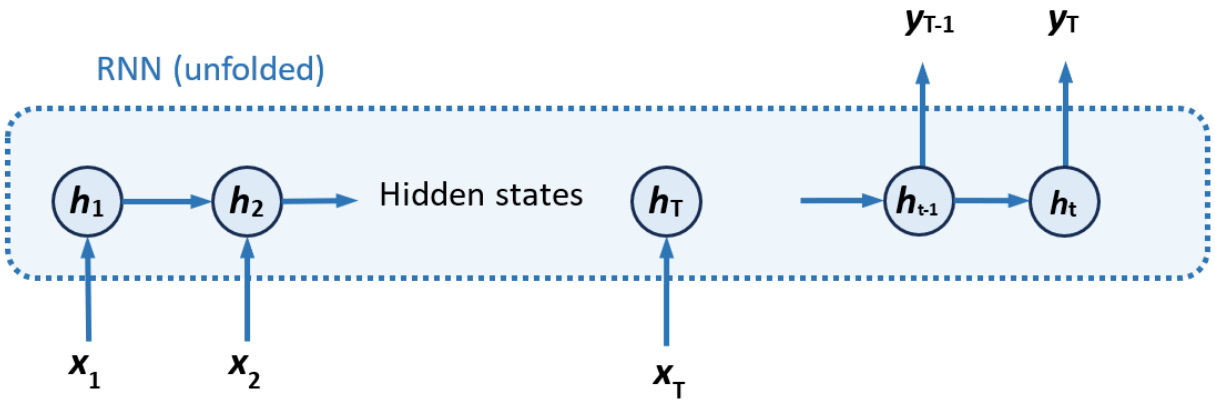
Recurrent Neural Networks

- The representation of a seq is learned by maintaining a hidden state vector capturing the current state of the seq
- The hidden state vector is computed from both the current input vector and the preceding hidden state vector
- Rightmost "unrolled" view of the RNN



RNN and Seq-to-Seq Model

- RNN can have multiple input-output combinations depending on the output(s) connected to the hidden layers (i.e., state/memory/thought layers)
  - such as one-to-one, one-to-many, many-to-one, and many-to-many
- We use indirect many-to-many or sequence-to-sequence (seq2seq), where an input sequence is transformed into another, different output sequence, not necessarily with the same length as the input

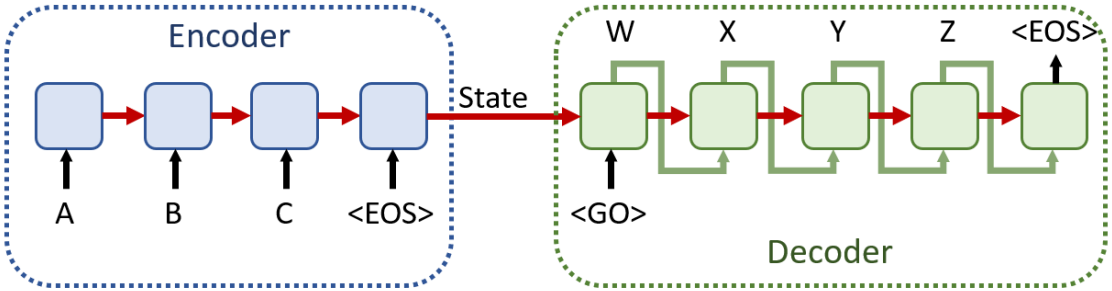


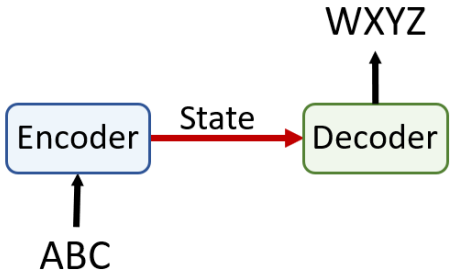
Encoder-Decoder Framework

- The input sequence `[A,B,C,<EOS>]` is decoded into the output sequence `[<GO>,W,X,Y,Z,<EOS>]`
- The model tries to maximize the following probability

$$P(y_1, \dots, y_{T_o} \mid x_1, \dots, x_T) = \prod_{t=1}^{T_o} P(y_t \mid \mathbf{v}, y_1, \dots, y_{t-1})$$

where $\mathbf{v}$ is the fixed length encoding of the input seq $\mathbf{x}$





Introduction to Transformer NN Models

- Transformers have revolutionized sequence modeling by self-attention mechanisms
- Designed to handle sequential data by capturing long-range dependencies
- **Self-Attention Mechanism** computes relationships between all elements in a sequence simultaneously
- **Positional Encoding** adds information about the position of tokens in the sequence and as a result compensates for the lack of inherent order in self-attention
- **Encoder-Decoder** maps a sequence to another sequence like an autoencoder
- Effectively captures relationships regardless of distance in the sequence
- More efficient than recurrent neural networks (RNNs) for large datasets
- Foundation for advanced models like BERT (bidirectional encoder representations from transformers), GPT (generative pre-trained transformer), etc.

The Need for Attention

- The decoder has to generate the entire output seq based solely on the $\mathbf{h}$ (i.e., state/memory/thought) vector
- The hidden vector has to encode all the information of the input sequence
- The encoder RNN hidden state will carry more info about the latest sequence elements than the earliest
- Using LSTM cells and reversing the input with a bidirectional model helps but cannot prevent it entirely

- It can be improved by iterating the **attention** from start to finish
- Attention mechanism can be applied sequentially (RNN), or can be applied in parallel (transformers)
  - The parallel application also helps unfold the RNN, thus speeding up iterative processing at the cost of higher memory usage
- The attention mechanism gives all words equal access to any part of a sentence in a faster parallel scheme and no longer suffers the wait time of serial processing

Seq-to-Seq with Attention

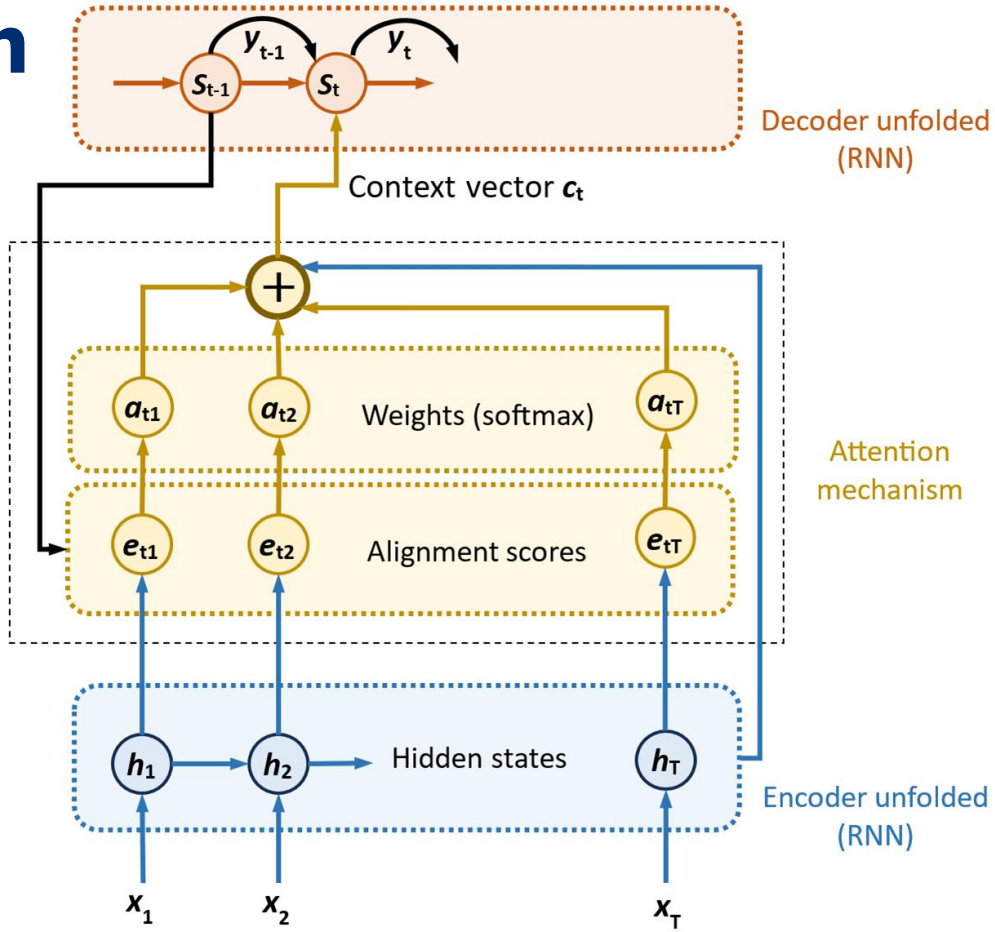
- Input seq token vectors $x_i \in \mathbb{R}^V$, $i = 1, \ldots, T$ where V is the embedding size and T is the sequence length
  - Input vocabulary size will be reduced to V, which is also used as the hidden layer size
    - The `nn.Embedding` layer in PyTorch can accomplish this
- The decoder has to generate the entire output sequence based solely on the thought vector, i.e., the encoder-hidden layer, $\mathbf{h}$
- The hidden decoder state s_t at time t is a function of the hidden state s_{t-1}, decoder output at step $t-1$ as y_{t-1}, and the context vector c_t
- Each decoder step has a unique context vector c_t
- The c_t for one decoder step is a weighted sum of all encoder $\mathbf{h}$
  - The encoder has access to all $\mathbf{h}$ at each output step t which removes the need to encode all source sequence information into a fixed-length vector

Bahdanau Attention

- c_t is the context vector for a D out step t out of T, the total output
- h_i is the hidden state of E step i out of T total input steps
- $\alpha_{t,i}$ is the scalar weight associated with h_i in the context of the current D step t

$$c_t = \sum_{i=1}^{T} \alpha_{t,i} h_i$$

- $\alpha_{t,i}$ is different for E and D steps



Bahdanau Attention Model

- As an example, given the input and output sequences with lengths of 10, the weights will be represented by a 10×10 matrix for a total of 100 weights
  - The attention mechanism will focus the attention of the D on different parts of the input sequence, depending on the current state of the output sequence
  - If $\alpha_{t,i}$ is large, then the decoder will pay a higher attention to h_i at step t
- The sum of all $\alpha_{t,i}$ for a D at step t is 1, might be a `softmax` operation

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{T} \exp(e_{t,j})}$$

- $e_{t,k}$ is an alignment model, which indicates how well the input sequence elements around position k match (or align with) the output at position t
  - This score (represented by the weight $\alpha_{t,i}$) is based on the previous decoder state s_{t-1} (s_t not computed yet), as well as the encoder state h_i

Bahdanau Attention Model

- $e_{t,i} = \mathbf{a}(s_{t-1}, h_i)$ where $\mathbf{a}$ is a differentiable function, which is trained with backpropagation together with the rest of the network
- Different functions satisfy these requirements, but the authors of the paper called it additive attention, which combines s_{t-1} and h_i with the help of addition. It can be,

$$e_{t,i} = \mathbf{a}(s_{t-1}, h_i) = v^{\mathrm{T}} \tanh(W[h_i; s_{t-1}])$$

$$e_{t,i} = \mathbf{a}(s_{t-1}, h_i) = v^{\mathrm{T}} \tanh(W_1 h_i + W_2 s_{t-1})$$

- The W is a weight matrix, applied over the concatenated vectors s_{t-1} and h_i, and v is a weight vector
- The alignment model can be represented as a simple feed-forward network with one hidden layer
- Additive attention provides **flexible scoring** and better for low-dimensional spaces

Luong Attention

- Several improvements added to Bahdanau attention such as the alignment scores e_t depend on the decoder hidden state s_t, as opposed to s_{t-1} in Bahdanau attention
- The Luong attention also proposed two types of multiplicative attention,

$$e_{t,i} = s_t^{\mathrm{T}} h_i$$

$$e_{t,i} = s_t^{\mathrm{T}} W_m h_i$$

- Note that the dot product acts as a similarity measure between vectors
- Multiplicative attention has more **computational efficiency** (optimized matrix multiplication, especially beneficial for large-scale data)

Hard Attention

- The hard and soft attention variants relate to the context vector c_t
- Hard attention computes the weights $\alpha_{t,i}$, and it only takes the hidden state h_{imax} with the maximum associated weight $\alpha_{t,imax}$
  - Then, the selected state h_{imax} serves as the context vector
  - Explicit selection of relevant inputs at the cost of computational complexity
- Hard attention limits it to a single state again, for a good reason in image recognition tasks, where the input sequence represents different regions of the same image
  - In such cases, choosing between multiple regions or a single region is better
- Unlike soft attention, hard attention is a stochastic process, which is non-differentiable

Local Attention

- Uses a window and has reduced computational complexity
  - $\mathcal{O}(n^2)$ versus $\mathcal{O}(n \cdot k)$, n is the sequence length
  - Local dependency modeling
- It represents a compromise between soft and hard attention
- Whereas these mechanisms take into account either all input hidden vectors (global) or just a single input vector, local attention takes a window of vectors surrounding a given input sequence location and then applies soft attention over this window only
- The center of the window position p_t (aka the aligned position) is picked according to the current output step t by assuming that the source and target sequences are roughly monotonically aligned

General Attention

- Attention is a general technique that can be applied to any seq2seq task
- Assuming hard attention, the vector s_{t-1} is a query executed against a database of key-value pairs, where the keys are s_t and the hidden states h_i are the values
  - These are often abbreviated as Q, K, and V matrices
- The keys Q and the values V of Luong and Bahdanau attention are the same vector semantically, where these attention models are Q and V
- The general attention mechanism uses all three components of Q, K, and V
- Hard attention, the query ($q = s_{t-1}$) against the database receives a single match, the key k_{imax} with the maximum weight $\alpha_{t,imax}$
- Hidden behind this key is the vector $v_{imax} = h_{imax}$, which is the actual value required
- The soft attention, where all values participate, can be the query results of all values with different weights

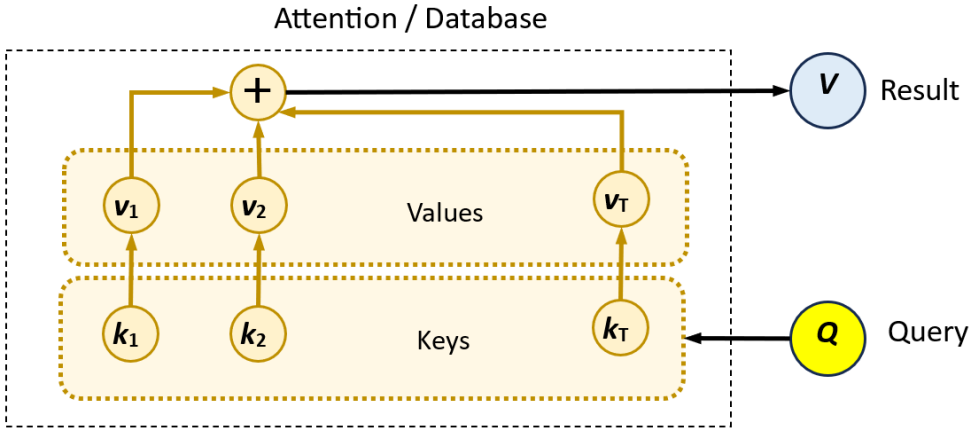
General Attention Model

- A generalized attention formula (based on the context vector c_t)

$$\alpha_{q,k_i} = \frac{\exp(e_{q,k_i})}{\sum_j \exp(e_{q,k_j})}$$

$$\text{Attention}(q, K, V) = \sum_i \alpha_{q,k_i} V_{k_i}$$

- In generic attention, the queries, keys, and vectors of the database are not necessarily related sequentially
  - The database not have to consist of the hidden RNN states at different steps but could contain any kind of information instead



Summary of Attention Mechanisms

Name	Score	Reference
Content-based	$\text{score}(\mathbf{s}_t, \mathbf{h}_i) = \cos[\mathbf{s}_t, \mathbf{h}_i]$	Graves14
Additive	$\text{score}(\mathbf{s}_t, \mathbf{h}_i) = v_a^{\mathrm{T}} \tanh(\mathbf{W}_a[\mathbf{s}_t, \mathbf{h}_i])$	Bahdanau15
Location-based	$\alpha_{t,i} = \text{softmax}(\mathbf{W}_a \mathbf{s}_t)$	Loung15
General	$\text{score}(\mathbf{s}_t, \mathbf{h}_i) = \mathbf{s}_t^{\mathrm{T}} \mathbf{W}_a \mathbf{h}_i$	Loung15
Dot-product	$\text{score}(\mathbf{s}_t, \mathbf{h}_i) = \mathbf{s}_t^{\mathrm{T}} \mathbf{h}_i$	Loung15
Scaled dot-product	$\text{score}(\mathbf{s}_t, \mathbf{h}_i) = \frac{\mathbf{s}_t^{\mathrm{T}} \mathbf{h}_i}{\sqrt{n}}$	Vaswani17

Multi-head Attention Mechanism

- Three matrices Q, K and V all derived from the original corpus, the input $\mathbf{X} \in \mathbb{R}^{d_x \times d_{model}}$ with d_x tokens in the seq and an embedding of size d_{model}
- The original length of the sequence is either d_x or truncated to d_x
- All three matrices Q (query θ), K (key Φ) and V (value g) are achieved multiplying the $\mathbf{X}$ matrix by Θ, Φ, and g, respectively
- The values (weights) of these matrices are learned by training

$$\mathrm{Attn}(Q, K, V) = V\mathrm{softmax}(\mathrm{score}(Q, K))$$

- Similar to NN training, Q, K, and V are initialized with zero mean and a low standard deviation normal
- Multi-head attention repeats this process for a k number of times and concatenates the output linearly to achieve parallelism
- If there are k heads then the output attention will be $Z = (z_0, z_1, \ldots, z_{k-1})$ partitioned over $\frac{d_{model}}{k}$, embedding dimension size

Transformer Example `Encoder`

```
class Encoder(nn.Module):
  def __init__(self, n_input, n_hidden):
    super().__init__()
    self.n_input, self.n_hidden = n_input, n_hidden
    # embedding layer - n_input=vocabulary size, n_hidden=will be embedding size
    self.embedding = nn.Embedding(self.n_input, self.n_hidden)
    # RNN layer, used token by token
    self.rnn_cell = nn.GRU(self.n_hidden, self.n_hidden, batch_first=True)

  # BLUE in the block diagram
  def forward(self, _X, _hn, bs=BATCH_SIZE):
    # single sequence is input
    # T x BS x hidden_size
    X_embedded = self.embedding(_X).view(bs, 1, -1)
    output, hn = self.rnn_cell(X_embedded, _hn)  # RNN iterations inside
    return output, hn
```

Decoder Constructor

```
class Decoder(nn.Module):
  def __init__(self, n_hidden, n_output, dropout=0.1):
    super().__init__()
    self.n_hidden, self.n_output = n_hidden, n_output

    self.embedding = nn.Embedding(self.n_output, self.n_hidden)
    self.dropout = nn.Dropout(dropout)
    self.attn = nn.Linear(in_features=self.n_hidden, out_features=self.n_hidden)
    self.w_c = nn.Linear(in_features=self.n_hidden*2, out_features=self.n_hidden)
    self.rnn_cell = nn.GRU(input_size=self.n_hidden, hidden_size=self.n_hidden,
batch_first=True)
    # output
    self.w_y = nn.Linear(in_features=self.n_hidden, out_features=self.n_output)
```

Decoder `forward` Method

```
def forward(self, _X, _hn, _enc_outs,
bs=BATCH_SIZE):
  X_embedded =
self.embedding(_X).view(bs,1,-1)
  X_embedded = self.dropout(X_embedded)
  T = _enc_outs.shape[1]
  # RED in the block diagram
  # hidden state, time t
  rnn_out, hn = self.rnn_cell(X_embedded,
_hn)
  # YELLOW in the block diagram
  if bs > 1:
    a1 = self.attn(hn)[0].view(1,-1)
    e1 = _enc_outs.transpose(1,2).reshape(-
1,T)
  else:
    a1 = self.attn(hn)[0]
    e1 = _enc_outs.t()
  alignment_scores = torch.mm(a1, e1)
  # weights
  attn_weights = softmax(alignment_scores,
dim=1)
  # multiplicative attent context vec c_t
  if bs > 1:
    e1 = _enc_outs.transpose(0,
1).reshape(T,-1)
    c_t = torch.mm(attn_weights,
e1).reshape(bs,-1)
  else:
    c_t = torch.mm(attn_weights, _enc_outs)
  # concatenate h_t and the context
  hidden_s_t = torch.cat([hn[0], c_t],
dim=1)
  # RED in the block diagram
  # hidden context
  hidden_s_t = tanh(self.w_c(hidden_s_t))
  # output
  output =
log_softmax(self.w_y(hidden_s_t), dim=1)
  return output, hn, attn_weights
```

Example General Attention

```
class Encoder(nn.Module):
  def __init__(self, n_inp, n_hid):
    super().__init__()
    self.n_inp, self.n_hid = n_inp, n_hid
    # n_inp=voc size, n_hid=embedding size
    self.emb = nn.Embedding(n_inp, n_hid)
    # RNN layer, used token by token
    self.rnn_cell = nn.GRU(n_hid, n_hid,
batch_first=True)

  def forward(self, _X, _hn, bs=BATCH_SIZE):
    # single seq input # T x BS x hidden_size
    X_embedded = self.emb(_X).view(bs, 1, -1)
    output, hn = self.rnn_cell(X_embedded, _hn)
    return output, hn

  def init_hidden(self, bs=BATCH_SIZE):
    return zeros(1, bs, self.n_hid, device=Dev)
```

```
class SingleHeadAttention(nn.Module):
  def __init__(self, n_hid):
    super().__init__()
    self.n_hid = n_hid
    # Linear layers to project input to Q, K, V
    self.Qlayer = nn.Linear(n_hid, n_hid)
    self.Klayer = nn.Linear(n_hid, n_hid)
    self.Vlayer = nn.Linear(n_hid, n_hid)

  def forward(self, _X):
    # X input embeddings BS x T x n_hid
    # returns context vectors BS x T x n_hid
    Q = self.Qlayer(_X)
    K = self.Klayer(_X)
    V = self.Vlayer(_X)
    attn_scores = matmul(Q, K.transpose(-2,-1))
/ sqrt(tensor(self.n_hid))
    attn_ws = F.softmax(attn_scores, dim=-1)
    context = matmul(attn_ws, V)
    return context, attn_ws
```

GLUE Benchmark for NLP Tasks

- The GLUE benchmark is a collection of datasets for evaluating the performance models for sentence understanding tasks differing size, text type, and difficulty levels
- **Single sentence tasks**
  - **CoLA** The Corpus of Linguistic Acceptability dataset - consists of English acceptability judgments drawn from articles on linguistic theory
  - **SST-2** The Stanford Sentiment Treebank dataset - includes sentences from movie reviews and human annotations of their sentiment with pos/neg labels
- **Similarity and paraphrase tasks**
  - **MRPC** The Microsoft Research Paraphrase Corpus dataset - examines whether the sentences in a pair are semantically equivalent
  - **QQP** The Quora Question Pairs dataset - decides whether a pair of questions is semantically equivalent
  - **STS-B** The Semantic Textual Similarity Benchmark dataset - collection of sentence pairs drawn from news headlines, with a similarity score between [1,5]

GLUE Benchmark

- **Inference tasks**
  - **MNLI** The Multi-Genre Natural Language Inference corpus - a collection of sentence pairs with textual entailment to predict if the text entails a hypothesis (entailment), contradicts the hypothesis (contradiction), or neither (neutral)
  - **QNLI** Question Natural Language Inference dataset - a converted version of SquAD to check whether a sentence contains the answer to a question
  - **RTE** The Recognizing Textual Entailment dataset - a task of textual entailment challenges combining data from various sources; similar to the previous QNLI dataset, where the task is to check whether a first text entails a second one
  - **WNLI** The Winograd Natural Language Inference schema challenge - a pronoun resolution task linking a pronoun and a phrase in a sentence

References

- Foster, David. Generative deep learning. " O'Reilly Media, Inc.", 2023.
- Raschka, Sebastian, et al. Machine Learning with PyTorch and Scikit-Learn: Develop machine learning and deep learning models with Python. Packt Publishing Ltd, 2022.
- Sutskever, Ilya, Oriol Vinyals, and Quoc V. Le. "Sequence to sequence learning with neural networks." Advances in neural information processing systems 27 (2014).
- Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. "Neural machine translation by jointly learning to align and translate." arXiv preprint arXiv:1409.0473 (2014).
- Vaswani, Ashish, et al. "Attention is all you need." Advances in neural information processing systems 30 (2017).
- Glue Datasets. https://huggingface.co/datasets/nyu-mll/glue. Accessed Oct 2024.

JOHNS HOPKINS

WHITING SCHOOL
of ENGINEERING

© The Johns Hopkins University 2025, All Rights Reserved.